CoconutAI

Coconut Disease Detector - System Description

> ***Plain-English version.*** *This document was rewritten from the current project source code. It only describes features found in the application. Where the code has a limit or an unfinished connection, that limit is stated clearly.*

1. System Description

CoconutAI is a web application for detecting coconut leaf conditions from images. The application is also called the Coconut Disease Detector in the backend code. A signed-in user can upload a coconut image or use the live camera screen. The system runs an AI model and shows the detected labels, confidence scores, and marked image.

Uploaded detections are saved as records. An Expert can review these records and mark them as verified or pending. When an Expert verifies a record, the system creates a notification for the owner of that record. The user can then see the verified record and its saved recommendations.

The application also has a disease map. It shows saved records that have valid image GPS coordinates. In the web interface, the Farmer view has the map while the Expert view hides the Farmer navigation.

2. Users and Access

User	What the current system lets the user do
Farmer	Sign in, upload images, use the live camera screen, view their detection records, see verification status, receive in-app notifications, view recommendations, use the map, and download a PDF report of their records.
Expert	Sign in to the Expert view, see records from users, filter pending and verified records, mark a record verified or pending, and save expert recommendations for a selected record. The Expert view also has an audit log.

The reviewed code has Farmer and Expert views. It does not show a separate Administrator role.

3. How the System Works

1. The user signs in with Firebase Authentication.

2. The user uploads a JPG, PNG, or WEBP image. The browser can also send camera frames to the live detection screen.

3. For an uploaded image, the server checks that it is a valid image, reads any available image metadata, and runs the active YOLO26 v6 model.

4. The system returns the detected labels, confidence scores, boxes around detections, and an annotated image.

5. A saved upload starts as **pending verification**. The AI score does not verify the result by itself.

6. The system stores the record and its related image files. It tries Firestore first, can fall back to Firebase Realtime Database, and keeps local SQLite storage for offline or fallback use.

7. An Expert can review a record. When the Expert marks it verified, the system saves the verifier details, creates a recommendation snapshot, and creates an in-app notification for the record owner.

8. The Farmer can view their records. The normal Farmer dashboard uses verified records for its main statistics.

Live camera and drone screen

The project includes a live WebSocket detection endpoint. The browser camera sends frames to the server, and the system returns annotated frames. When higher-confidence detections are found during a live session, the system can write a drone-log entry with the current GPS position when one is available. This is a live camera feature; it does not prove that a physical drone is connected or flying.

4. Detection Labels

The label returned by an image is taken from the loaded **weights.pt** model. The exact list should therefore be checked from the running model before a final research paper claims a fixed class list.

The Expert recommendation editor currently lists: Caterpillars, Drying of Leaflets, Healthy, Pestalotiopsis, Bud Rot, and Stem Bleeding. The user interface highlights Healthy, Stem Bleeding, Pestalotiopsis, and Bud Rot on its map and filters.

Important: the current detector code filters out old labels named Cercospora and Leaf Rot. It also states that the loaded detection model must contain Stem Bleeding training examples before the detector itself can return Stem Bleeding. For this reason, this document does not claim that every label in the Expert editor is already an active model output.

5. Recommendations and Verification

The system can provide fertilizer, treatment, and prevention recommendations. The code first tries to use saved machine-learning recommendation files. If those files cannot be used, it uses built-in recommendation content. An Expert can also save a custom recommendation for one selected record.

The current Expert verification endpoint changes the record status between pending and verified. It does **not** include a field for changing the AI detection label. Therefore, this system description does not claim that the Expert can correct a predicted disease class.

6. GPS and Map

- For an uploaded image, the server uses valid GPS data embedded in the image, including DJI EXIF data when available.

- If an uploaded image has no usable embedded GPS data, that upload is left unmapped. Browser or form coordinates are not used as a replacement for an uploaded image.

- The app can keep a farm center after a valid image GPS location is saved.

- The web map uses MapLibre GL. It shows location pins, a heatmap, and an overview map for records with location data.

- The live camera screen can use a current GPS position for live markers and drone-log entries when GPS is available.

7. Main Technologies Used

Technology	Use in this project
Python and FastAPI	Backend web API, image upload, detection, records, reporting, and WebSocket streaming.
YOLO26 v6 and Ultralytics	The active detector loads local weights.pt and uses the Ultralytics backend for normal predictions.
OpenVINO	The project contains OpenVINO code and dependency support, but the active detector is set to the Ultralytics backend in the current model code.
Firebase	Authentication plus cloud storage paths for Firestore and Realtime Database.
SQLite	Local record storage for offline or fallback use.
HTML, CSS, and JavaScript	The browser interface for login, upload, records, recommendations, notifications, and live camera controls.
MapLibre GL	The browser map, map pins, heatmap, and overview map.
ReportLab	PDF report generation for a user's detection records.

8. Input, Process, and Output

Input	Process	Output
Signed-in user details; uploaded JPG, PNG, or WEBP images; live camera frames; image metadata; available GPS data; Expert status and recommendation details.	Authenticate the user; validate the image or stream frame; run AI detection; create annotated output; save records; allow Expert review; save a verification status and recommendation snapshot; build map data when coordinates exist.	Detected labels, confidence scores, marked images, saved records, pending or verified status, recommendations, notifications, GPS map pins, CSV exports, and a PDF report for the user's records.

9. System Limits

- The AI result depends on the image quality, lighting, visible symptoms, training images, and model weights.

- The model can only return labels that exist in the loaded model. The exact model labels should be checked before they are listed in final academic documents.

- An AI result is not automatically verified. Uploaded records remain pending until an Expert verifies them.

- The current code lets an Expert verify a record and save recommendations, but it does not provide a way to edit the AI disease label.

- Mapping depends on valid GPS information. Uploaded images without usable embedded coordinates are not placed on the map.

- Cloud saving depends on Firebase availability. The application has Realtime Database and local SQLite fallback paths, but a deployment must still have correct Firebase settings.

- The live camera feature depends on camera permission, browser support, network connection, server capacity, and a valid signed-in session.

- Recommendations are generated from configured machine-learning files, built-in content, or an Expert's saved record recommendation. They are support information, not a substitute for professional agricultural advice.

10. Short Summary

CoconutAI is a web-based coconut disease detection and record-management system. It uses a local YOLO26 v6 model to analyze uploaded images and live camera frames. Users can keep their detection records, view verified results, see available recommendations, receive verification notifications, and view GPS-tagged records on a map. Experts can review records, mark them verified or pending, and save recommendations for a selected record. The system supports decision-making, but its AI results and recommendations should be checked by a qualified agricultural professional.